

Assignment 4: Regularization

Q1. Please answer the following questions in your own words.

1. What is the intuition of adding a penalty to mean squared error, that grows in the "size" (absolute or squared value) of the model parameters?
2. How does regularization provide a way of exploring the bias-variance trade-off?
3. What is the difference between LASSO and Ridge regression? How do the answers typically change for the two problems?
4. How do we typically scale variables for use in regularized regression? Why?
5. How is the penalty α typically selected?
6. When conducting cross validation, do you include the penalty term in evaluating the cross validated MSE? Why or why not?

#1

When model parameters get very large, it means the model is overfitting, meaning it is memorizing the training data too closely instead of learning general patterns. Adding a penalty makes the model pay a price for having big parameters, so its forced to stay simple and work better on new data.

#2

When you increase the penalty (regularization), the model gets simpler and more biased (it can not fit the data as closely), but variance drops because it stops overreacting to noise in the training data. So by turning the regularization knob up or down, you're directly controlling where you land on the bias-variance tradeoff.

#3

Both add a penalty to keep parameters small, but Ridge shrinks all parameters a little bit, while LASSO can shrink some parameters all the way to zero (effectively removing those features entirely). This means Ridge keeps all variables in the model, while LASSO does automatic feature selection by kicking out the least useful ones.

#4

We standardize all variables to have a mean of zero and a standard deviation of one before running regularized regression. This is because the penalty treats all parameters equally, so if one variable is measured in thousands and another in decimals, the penalty would unfairly punish the smaller-scaled variable's coefficient.

#5

The penalty is typically selected by using cross-validation, where you try a bunch of different alpha values and see which ones makes the model perform best on held out data. You pick the alpha that gives you the lowest average error across those validation sets.

#6

No, you only use regular MSE (no penalty) to evaluate performance on the validation fold, because the penalty is just a training tool to keep the model simple, not an actual measure of how well your predictions are. You want to evaluate how good your predictions truly are on new data, and the penalty term isn't part of that.

Q2. This is a case study on regularization.

1. Import the `cars_hw.csv` dataset. Create an `Age` variable for each vehicle. Take `Mileage_Run` and `Age`, and (a) use `PolynomialFeatures` to create a third degree expansion, (b) use `StandardScaler` to z-score normalize them.
2. Use your features, run linear regression. What is the sign for the interaction between `Mileage_Run` and `Age`?
3. Use `LassoCV` to regularize your linear regression, using 20-fold cross validation. (Hint: I used the grid `alphas = np.logspace(1,3,20)` to find the cost parameter)
4. Plot the cross-validated MSE by α .
5. Plot the coefficient paths by α .
6. Which features are actually selected? What proportion are set equal to zero?
7. Compare the linear regressions and optimally regularized coefficients. Do any coefficients increase in magnitude from linear regression to LASSO? Do any change sign?

#1

```
In [6]: from sklearn.preprocessing import PolynomialFeatures, StandardScaler
import pandas as pd
cars = pd.read_csv('cars_hw.csv')

cars['Age'] = 2025 - cars['Make_Year']

X = cars[['Mileage_Run', 'Age']]
poly = PolynomialFeatures(degree=3)
X_poly = poly.fit_transform(X)

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_poly)
```

#2

```
In [9]: from sklearn.linear_model import LinearRegression

y = cars['Price']

lr = LinearRegression()
lr.fit(X_scaled, y)

feature_names = poly.get_feature_names_out(['Mileage_Run', 'Age'])
coef_df = pd.DataFrame({'Feature': feature_names, 'Coefficient': lr.coef_})
print(coef_df)
```

	Feature	Coefficient
0	1	0.000000e+00
1	Mileage_Run	7.861191e+05
2	Age	-2.141229e+05
3	Mileage_Run^2	-1.783803e+05
4	Mileage_Run Age	-1.453125e+06
5	Age^2	2.789820e+05
6	Mileage_Run^3	-6.514490e+04
7	Mileage_Run^2 Age	3.109725e+05
8	Mileage_Run Age^2	6.186680e+05
9	Age^3	-1.724494e+05

What is the sign for the interaction between Mileage_Run and Age?

The interaction term between Mileage_Run and Age is negative, meaning that cars that are both high mileage and old are penalized extra in price. The two factors together hurt the price more than each one individually.

#3

```
In [17]: import warnings
from sklearn.exceptions import ConvergenceWarning
import numpy as np
from sklearn.linear_model import LassoCV

alphas = np.logspace(1, 3, 20)

with warnings.catch_warnings():
    warnings.simplefilter("ignore", ConvergenceWarning)
    lasso = LassoCV(alphas=alphas, cv=20, max_iter=10000)
    lasso.fit(X_scaled, y)

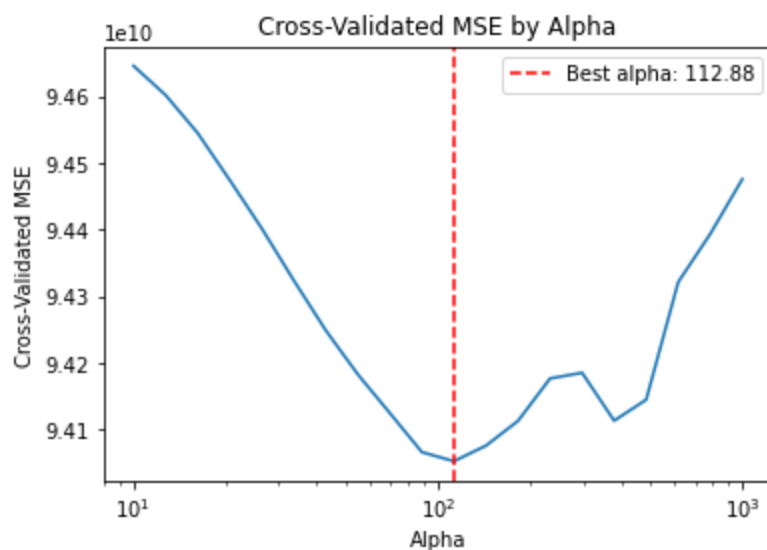
print("Best alpha:", lasso.alpha_)
```

Best alpha: 112.88378916846884

#4

```
In [18]: import matplotlib.pyplot as plt

plt.plot(lasso.alphas_, lasso.mse_path_.mean(axis=1))
plt.xscale('log')
plt.xlabel('Alpha')
plt.ylabel('Cross-Validated MSE')
plt.title('Cross-Validated MSE by Alpha')
plt.axvline(lasso.alpha_, color='red', linestyle='--', label=f'Best alpha: {lasso.alpha_}')
plt.legend()
plt.show()
```



#5

```
In [21]: from sklearn.linear_model import lasso_path

with warnings.catch_warnings():
    warnings.simplefilter("ignore", ConvergenceWarning)
```

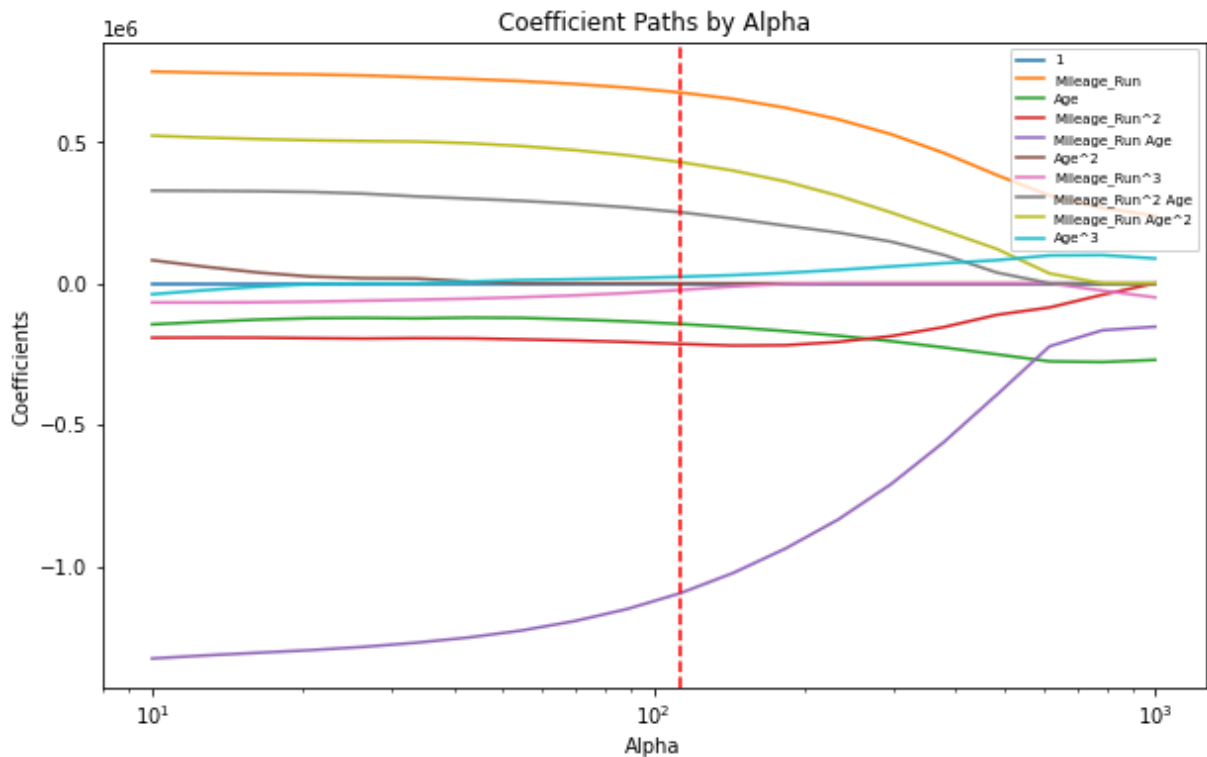
```

alphas_path, coefs_path, _ = lasso_path(X_scaled, y, alphas=alphas)

feature_names = poly.get_feature_names_out(['Mileage_Run', 'Age'])

plt.figure(figsize=(10, 6))
plt.plot(alphas_path, coefs_path.T)
plt.xscale('log')
plt.xlabel('Alpha')
plt.ylabel('Coefficients')
plt.title('Coefficient Paths by Alpha')
plt.legend(feature_names, loc='upper right', fontsize=7)
plt.axvline(lasso.alpha_, color='red', linestyle='--', label=f'Best alpha: {lasso.alpha_}')
plt.show()

```



#6

LASSO selected 7 out of 10 features, keeping Mileage_Run, Age, Mileage_Run², Mileage_Run Age, Mileage_Run³, Mileage_Run² Age, Mileage_Run Age², and Age³. 3 out of 10 features were set to zero, the intercept term Age² and the constant term 1.

#7

Comparing the linear regression and LASSO coefficients, no coefficients change sign, and all coefficients maintain the

same positive or negative direction in both models. However, all coefficients decrease in magnitude under LASSO due to the regularization penalty shrinking them. The Mileage_Run Age interaction term shrinks the most dramatically, going from -1,453,125 in linear regression down to -1,151,929 in LASSO.

Q3. This is a case study on regularization.

1. Import the `heart_failure_clinical_records_dataset.csv` dataset. Use `PolynomialFeatures` to create a third-degree expansion of `age`, `ejection_fraction`, and `serum_creatinine`, and then use `StandardScaler` to z-score normalize your results. Use `PolynomialFeatures` with `interaction_only=True` to interact the dummy/categorical variables `anaemia`, `diabetes`, `high_blood_pressure`, and `smoking`. Concatenate these results into your feature/covariate matrix.
2. Use your features, run linear regression. Are there any sign patterns that appear counterintuitive? Why? Can you see how the inclusion of higher-order powers or interactions might resolve the apparent contradiction?
3. Use `LassoCV` to regularize your linear regression, using 20-fold cross validation. (Hint: I used the grid `alphas = np.logspace(-5, 5, 30)` to find the cost parameter)
4. Plot the cross-validated MSE by α .
5. Plot the coefficient paths by α .
6. Which features are actually selected? What proportion are set equal to zero? Compare the linear regressions and optimally regularized coefficients. Do any coefficients increase in magnitude from linear regression to LASSO? Do any change sign? Do the sign patterns for the linear_model or the Lasso seem to make more sense? Explain why this might be the case from the perspective of the bias-variance trade-off.

#1

```
In [25]: import numpy as np
from sklearn.preprocessing import PolynomialFeatures, StandardScaler

heart = pd.read_csv('heart_failure_clinical_records_dataset.csv')

X_cont = heart[['age', 'ejection_fraction', 'serum_creatinine']]
poly_cont = PolynomialFeatures(degree=3)
X_cont_poly = poly_cont.fit_transform(X_cont)
scaler = StandardScaler()
X_cont_scaled = scaler.fit_transform(X_cont_poly)

X_cat = heart[['anaemia', 'diabetes', 'high_blood_pressure', 'smoking']]
poly_cat = PolynomialFeatures(degree=3, interaction_only=True)
X_cat_poly = poly_cat.fit_transform(X_cat)
```

**Concatenate these results into your feature/covariate matrix.
(Done Below)**

```
In [26]: X_combined = np.hstack([X_cont_scaled, X_cat_poly])
         print(X_combined.shape)
```

(299, 35)

#2

```
In [28]: import warnings
         from sklearn.linear_model import LinearRegression

         y_heart = heart['DEATH_EVENT']

         with warnings.catch_warnings():
             warnings.simplefilter("ignore")
             lr_heart = LinearRegression()
             lr_heart.fit(X_combined, y_heart)
```

**Are there any sign patterns that appear counterintuitive?
Why? Can you see how the inclusion of higher-order powers
or interactions might resolve the apparent contradiction?**

```
In [29]: feature_names_cont = poly_cont.get_feature_names_out(['age', 'ejection_fraction', '
         feature_names_cat = poly_cat.get_feature_names_out(['anaemia', 'diabetes', 'high_blood_pressure', '
         all_feature_names = np.concatenate([feature_names_cont, feature_names_cat])

         coef_df_heart = pd.DataFrame({'Feature': all_feature_names, 'Coefficient': lr_heart.coef_})
         print(coef_df_heart)
```

	Feature	Coefficient
0	1	6.527673e+12
1	age	1.582605e+00
2	ejection_fraction	-1.985344e+00
3	serum_creatinine	-7.718981e-01
4	age^2	-3.285949e+00
5	age ejection_fraction	-1.318689e+00
6	age serum_creatinine	1.661941e+00
7	ejection_fraction^2	3.370727e+00
8	ejection_fraction serum_creatinine	2.860618e+00
9	serum_creatinine^2	-2.791914e+00
10	age^3	1.947969e+00
11	age^2 ejection_fraction	4.872666e-01
12	age^2 serum_creatinine	-8.874297e-01
13	age ejection_fraction^2	1.022706e+00
14	age ejection_fraction serum_creatinine	-1.204560e+00
15	age serum_creatinine^2	1.389150e+00
16	ejection_fraction^3	-1.653912e+00
17	ejection_fraction^2 serum_creatinine	-1.540637e+00
18	ejection_fraction serum_creatinine^2	1.029053e+00
19	serum_creatinine^3	2.136533e-01
20	1	-4.440892e-15
21	anaemia	1.831042e-02
22	diabetes	-2.887416e-01
23	high_blood_pressure	-1.103338e-01
24	smoking	-3.085440e-01
25	anaemia diabetes	1.110088e-01
26	anaemia high_blood_pressure	3.021799e-02
27	anaemia smoking	1.786319e-01
28	diabetes high_blood_pressure	3.777022e-01
29	diabetes smoking	6.360028e-01
30	high_blood_pressure smoking	3.594928e-01
31	anaemia diabetes high_blood_pressure	-1.741408e-01
32	anaemia diabetes smoking	-2.692260e-01
33	anaemia high_blood_pressure smoking	-3.824025e-01
34	diabetes high_blood_pressure smoking	-3.671496e-01

A counterintuitive sign pattern is that `ejection_fraction` has a negative coefficient (-1.99), which seems wrong since higher ejection fraction means a healthier heart and should reduce death risk. However, higher-order terms like `ejection_fraction`² (+3.37) and `ejection_fraction`³ (-1.65) show that the relationship is actually nonlinear and curves back, meaning the simple negative sign alone is misleading. Also, `diabetes` alone is negative (-0.29) suggesting it reduces death risk, but its interactions like `diabetes smoking` (+0.64) and `diabetes high_blood_pressure` (+0.38) reveal that `diabetes` only becomes dangerous in combination with other conditions.

#3

```
In [30]: from sklearn.linear_model import LassoCV
import warnings

alphas_heart = np.logspace(-5, 5, 30)
```



```

with warnings.catch_warnings():
    warnings.simplefilter("ignore")
    lasso_heart = LassoCV(alphas=alphas_heart, cv=20, max_iter=10000)
    lasso_heart.fit(X_combined, y_heart)

print("Best alpha:", lasso_heart.alpha_)

```

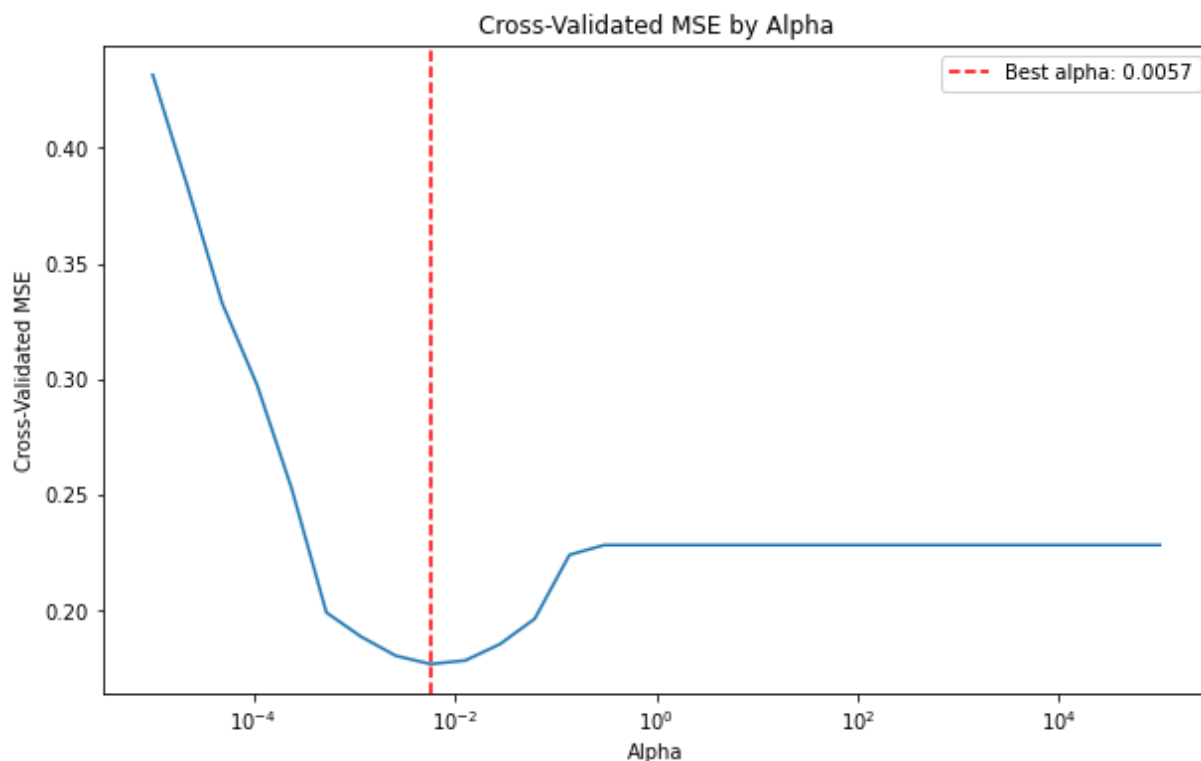
Best alpha: 0.005736152510448681

#4

```

In [31]: with warnings.catch_warnings():
          warnings.simplefilter("ignore")
          plt.figure(figsize=(10, 6))
          plt.plot(lasso_heart.alphas_, lasso_heart.mse_path_.mean(axis=1))
          plt.xscale('log')
          plt.xlabel('Alpha')
          plt.ylabel('Cross-Validated MSE')
          plt.title('Cross-Validated MSE by Alpha')
          plt.axvline(lasso_heart.alpha_, color='red', linestyle='--', label=f'Best alpha')
          plt.legend()
          plt.show()

```



#5

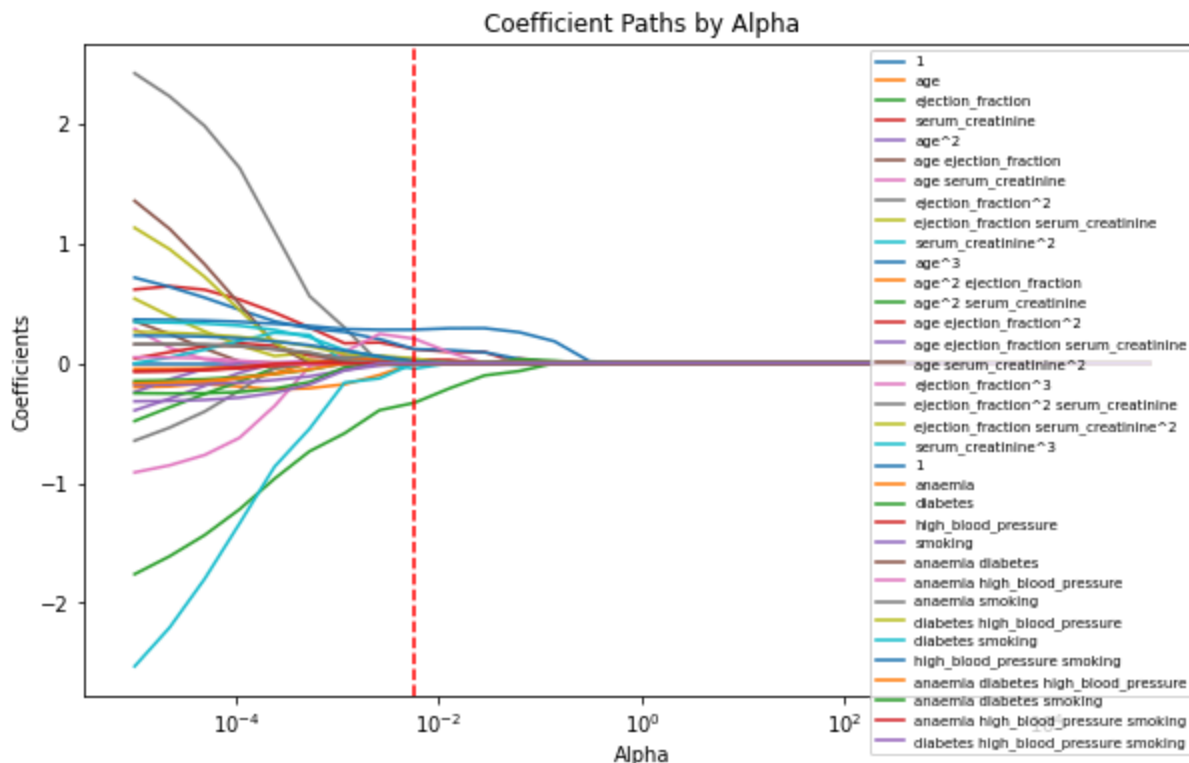
```

In [32]: with warnings.catch_warnings():
          warnings.simplefilter("ignore")
          alphas_path_heart, coefs_path_heart, _ = lasso_path(X_combined, y_heart, alphas_heart)

          plt.figure(figsize=(10, 6))

```

```
plt.plot(alphas_path_heart, coefs_path_heart.T)
plt.xscale('log')
plt.xlabel('Alpha')
plt.ylabel('Coefficients')
plt.title('Coefficient Paths by Alpha')
plt.legend(all_feature_names, loc='upper right', fontsize=7)
plt.axvline(lasso_heart.alpha_, color='red', linestyle='--', label=f'Best alpha: {1
plt.show()
```



#6

```
In [34]: lasso_coef_df_heart = pd.DataFrame({'Feature': all_feature_names, 'Coefficient': la
print(lasso_coef_df_heart)
print("\nNumber of zero coefficients:", (lasso_heart.coef_ == 0).sum())
print("Total features:", len(lasso_heart.coef_))
```

	Feature	Coefficient
0	1	0.000000
1	age	-0.000000
2	ejection_fraction	-0.329656
3	serum_creatinine	0.116875
4	age^2	0.000000
5	age ejection_fraction	-0.000000
6	age serum_creatinine	0.000000
7	ejection_fraction^2	0.000000
8	ejection_fraction serum_creatinine	0.000000
9	serum_creatinine^2	-0.000000
10	age^3	0.117992
11	age^2 ejection_fraction	0.000000
12	age^2 serum_creatinine	0.000000
13	age ejection_fraction^2	0.000000
14	age ejection_fraction serum_creatinine	0.000000
15	age serum_creatinine^2	-0.000000
16	ejection_fraction^3	0.204822
17	ejection_fraction^2 serum_creatinine	0.032801
18	ejection_fraction serum_creatinine^2	-0.000000
19	serum_creatinine^3	-0.043266
20	1	0.000000
21	anaemia	0.014178
22	diabetes	0.000607
23	high_blood_pressure	0.013608
24	smoking	-0.000000
25	anaemia diabetes	0.000000
26	anaemia high_blood_pressure	0.000000
27	anaemia smoking	0.000000
28	diabetes high_blood_pressure	0.053968
29	diabetes smoking	0.003466
30	high_blood_pressure smoking	0.017805
31	anaemia diabetes high_blood_pressure	0.000000
32	anaemia diabetes smoking	0.000000
33	anaemia high_blood_pressure smoking	0.000000
34	diabetes high_blood_pressure smoking	0.000000

Number of zero coefficients: 23

Total features: 35

Which features are actually selected? What proportion are set equal to zero?

LASSO selected only 12 out of 35 features, setting 23 features to zero and throwing out most of the higher order and interaction terms. The key features kept were ejection_fraction, serum_creatinine, age³, ejection_fraction³, ejection_fraction² serum_creatinine, serum_creatinine³, anaemia, diabetes, high_blood_pressure, diabetes high_blood_pressure, diabetes smoking, and high_blood_pressure smoking.

Do any coefficients increase in magnitude from linear regression to LASSO? Do any change sign? Do the sign patterns for the linear_model or the Lasso seem to make more

sense? Explain why this might be the case from the perspective of the bias-variance trade-off.

Compared to linear regression, almost every coefficient shrank in size, for example `ejection_fraction` went from about 1.99 down to 0.33 and `serum_creatinine` went from 0.77 down to 0.12. Some coefficients also flipped signs, most importantly `serum_creatinine` flipped from negative in linear regression to positive in LASSO, which now actually makes medical sense because higher serum creatinine means your kidneys are failing. `ejection_fraction`³ also flipped from negative to positive, which makes sense because the cubic term captures the nonlinear curve of how ejection fraction affects death risk. The linear regression signs were often wrong and the numbers were huge because the model was overfitting and memorizing the training data instead of learning real patterns. LASSO signs make more real world sense because the penalty forced the model to stay simple and only keep the most meaningful relationships. From a bias variance perspective, linear regression had very low bias but way too much variance, while LASSO accepts a tiny bit more bias in exchange for a much more stable model.

Q4. To better understand the math of regularization, we'll solve the regularized linear model problem with a single explanatory variable. So, the model is

$$\tilde{y}_i = \tilde{b}_0 + \tilde{b}_1 \tilde{x}_i,$$

where

$$\tilde{y}_i = y_i - \bar{y} \quad \text{and} \quad \tilde{x}_i = x_i - \bar{x}.$$

Recall, we do this mean-normalization of x and y , because

$$\frac{1}{n} \sum_{i=1}^n \tilde{y} = \frac{1}{n} \sum_{i=1}^n y_i - \bar{y} = 0,$$

and likewise for x . This trick makes the calculations easier and the results more easily interpretable.

1. To do ridge regression, add a penalty $+\alpha(b_1)^2$ to mean squared error. What is the objective function for this problem?
2. Take the derivatives of your objective function with respect to b_0 and b_1 . Set these equations equal to zero. Solve the two equations in two unknowns for b_1 and b_0 .
3. How does increasing α change the slope coefficient?
4. If we instead used the LASSO/L1 penalty, $+\alpha|b_1|$, what challenge do you run into? This is conceptually difficult, but take 5 minutes and try to figure out the solution, and in particular, when is it optimal to set $b_1 = 0$?

#1

The objective function is the thing we are trying to minimize, and for ridge regression we take the regular mean squared error and add a penalty term on top of it, giving us -->

$$\frac{1}{n} \sum_{i=1}^n (\tilde{y}_i - \tilde{b}_0 - \tilde{b}_1 \tilde{x}_i)^2 + \alpha(b_1)^2$$

This means we are trying to find the value of b_1 that makes our predictions accurate (low MSE) while also keeping b_1 small (low penalty)

#2

 Ridge Derivation

Minimize

$$\frac{1}{n} \sum_{i=1}^n (\tilde{y}_i - b_1 \tilde{x}_i)^2 + \alpha b_1^2$$

Take derivative w/ respect to $b_1 \rightarrow$ set equal to zero

$$\frac{\partial}{\partial b_1} \left[\frac{1}{n} \sum_{i=1}^n (\tilde{y}_i - b_1 \tilde{x}_i)^2 + \alpha b_1^2 \right] = 0$$

Using Chain Rule:

$$-\frac{2}{n} \sum_{i=1}^n \tilde{x}_i (\tilde{y}_i - b_1 \tilde{x}_i) + 2\alpha b_1 = 0$$

$$-\frac{1}{n} \sum_{i=1}^n \tilde{x}_i (\tilde{y}_i - b_1 \tilde{x}_i) + \alpha b_1 = 0$$

$$-\frac{1}{n} \sum_{i=1}^n \tilde{x}_i \tilde{y}_i + \frac{b_1}{n} \sum_{i=1}^n \tilde{x}_i^2 + \alpha b_1 = 0$$

$$\frac{b_1}{n} \sum_{i=1}^n \tilde{x}_i^2 + \alpha b_1 = \frac{1}{n} \sum_{i=1}^n \tilde{x}_i \tilde{y}_i$$

$$b_1 \left(\frac{1}{n} \sum_{i=1}^n \tilde{x}_i^2 + \alpha \right) = \frac{1}{n} \sum_{i=1}^n \tilde{x}_i \tilde{y}_i$$

$$b_1 = \frac{\frac{1}{n} \sum_{i=1}^n \tilde{x}_i \tilde{y}_i}{\frac{1}{n} \sum_{i=1}^n \tilde{x}_i^2 + \alpha}$$

$$b_0 = \bar{y} - b_1 \bar{x}$$

#3

As alpha increases, the denominator gets bigger, which makes b_1 smaller. This means increasing alpha shrinks the slope coefficient closer and closer to zero, which is exactly the point of regularization, forcing the model to stay simple.

#4

The Lasso Function

$$\frac{1}{n} \sum_{i=1}^n (\tilde{y}_i - b_1 \tilde{x}_i)^2 + \alpha |b_1|$$

→ absolute value $|b_1|$ is not differentiable at $b_1 = 0$

For $b_1 > 0$

$$-\frac{1}{n} \sum_{i=1}^n \tilde{x}_i \tilde{y}_i + \frac{b_1}{n} \sum_{i=1}^n \tilde{x}_i^2 + \alpha = 0$$

For $b_1 < 0$

$$-\frac{1}{n} \sum_{i=1}^n \tilde{x}_i \tilde{y}_i + \frac{b_1}{n} \sum_{i=1}^n \tilde{x}_i^2 - \alpha = 0$$

Optimal to set $b_1 = 0$ when:

$$\left| \frac{1}{n} \sum_{i=1}^n \tilde{x}_i \tilde{y}_i \right| \leq \alpha$$

 LASSO Derivation

The challenge with LASSO is that the absolute value penalty has a sharp corner at zero meaning we can not take a normal derivative and set it equal to zero like we did with the ridge. Instead we have to think about it differently, by checking whether the correlation between x and y is strong enough to overcome the penalty α . If the correlation between x and y is very small and α is large enough, specifically when α is bigger than the absolute value of $\frac{1}{n} \sum x_i y_i$, then it is actually best to just set b_1 equal to zero entirely. This is exactly why LASSO can shrink coefficients all the way to zero while ridge can only shrink them very close to zero but never actually reach it.